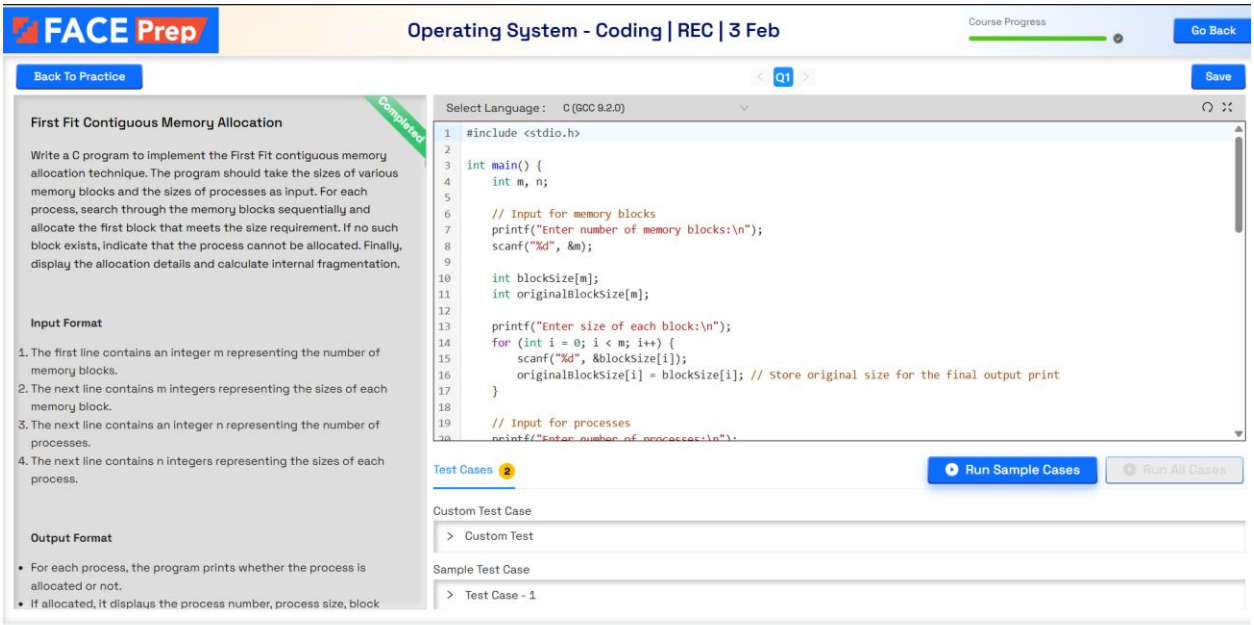
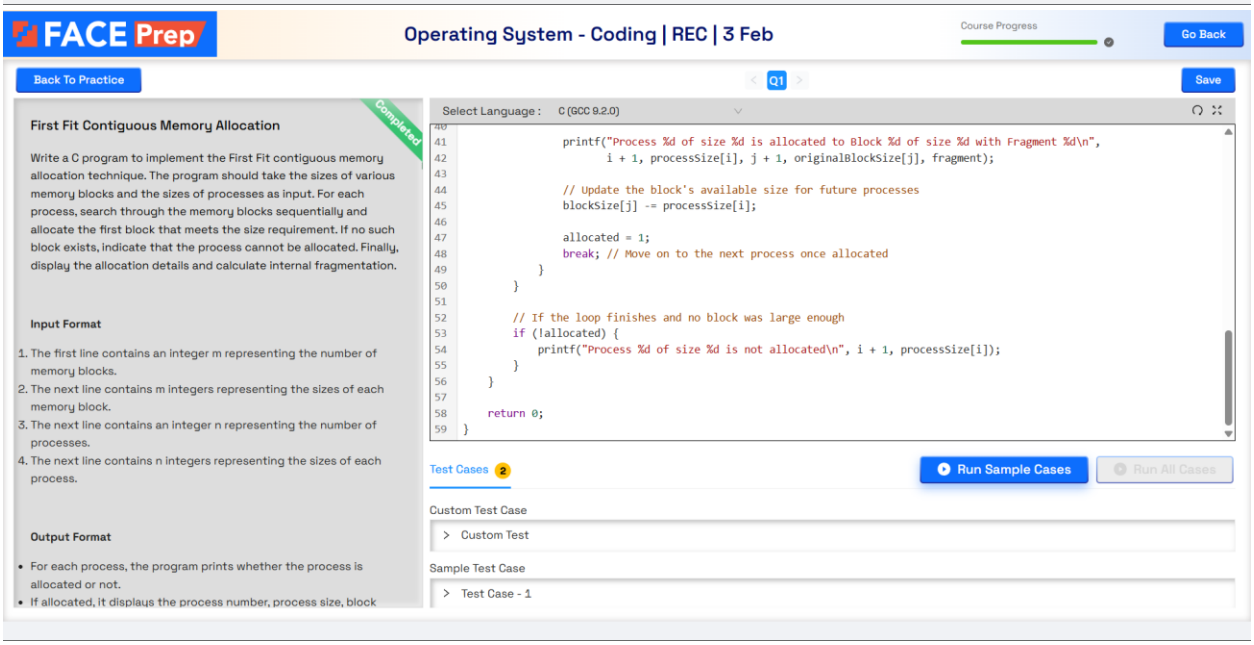

FACE Prep
Operating System - Coding | REC | 3 Feb
Course Progress
Go Back
Back To Practice
Q1
Save
Completed
First Fit Contiguous Memory Allocation
Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.
Input Format
1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.
Output Format
• For each process, the program prints whether the process is allocated or not.
• If allocated, it displays the process number, process size, block
Select Language : C (GCC 9.2.0)
#include <stdio.h>
int main() {
int m, n;
// Input for memory blocks
printf("Enter number of memory blocks:\n");
scanf("%d", &m);
int blockSize[m];
int originalBlockSize[m];
printf("Enter size of each block:\n");
for (int i = 0; i < m; i++) {
scanf("%d", &blockSize[i]);
originalBlockSize[i] = blockSize[i]; // Store original size for the final output print
}
// Input for processes
Test Cases 2
Run Sample Cases
Run All Cases
Custom Test Case
Custom Test
Sample Test Case
Test Case - 1


FACE Prep
Operating System - Coding | REC | 3 Feb
Course Progress
Go Back
Back To Practice
Q1
Save
Completed
First Fit Contiguous Memory Allocation
Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.
Input Format
1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.
Output Format
• For each process, the program prints whether the process is allocated or not.
• If allocated, it displays the process number, process size, block
Select Language : C (GCC 9.2.0)
printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
i + 1, processSize[i], j + 1, originalBlockSize[j], fragment);
// Update the block's available size for future processes
blockSize[j] -= processSize[i];
allocated = 1;
break; // Move on to the next process once allocated
}
}
// If the loop finishes and no block was large enough
if (!allocated) {
printf("Process %d of size %d is not allocated\n", i + 1, processSize[i]);
}
}
return 0;
}
Test Cases 2
Run Sample Cases
Run All Cases
Custom Test Case
Custom Test
Sample Test Case
Test Case - 1